

编译器导论

Chapter 3-1: Finite Automata

有限自动机

基于南京大学计算机学院课程讲义

时清凯 | qingkaishi@nju.edu.cn

2026 年 6 月 10 日

目录

前言	2
1 第一部分：数学预备知识 (Math Preliminaries)	3
1.1 字母表与字符串 (Alphabet and Strings)	3
1.2 字符串操作 (String Operations)	3
1.2.1 连接 (Concatenation)	3
1.2.2 反转 (Reverse)	3
1.2.3 长度 (Length)	3
1.3 空字符串与子字符串	4
1.4 幂、Kleene 星号与加号	4
1.5 语言上的操作 (Operations on Languages)	5
1.5.1 集合操作	5
1.5.2 连接、幂与闭包	5
2 第二部分：确定型有限自动机 (DFA)	6
2.1 有限自动机的直观理解	6
2.2 DFA 的形式化定义	6
2.2.1 扩展转移函数	7
2.3 DFA 定义的语言	7
2.3.1 DFA 的实现：字符串检查器	7
2.4 DFA 最小化 (DFA Minimization)	8
2.4.1 最小化算法	8
2.4.2 最小化结果	9
2.5 DFA 双向模拟 (DFA Bi-Simulation)	9
3 第三部分：非确定型有限自动机 (NFA)	10
3.1 NFA 的直观理解	10
3.2 ϵ 转移 (ϵ -Transition)	10
3.3 NFA 的形式化定义	10
3.4 ϵ -闭包 (ϵ -Closure)	11
3.5 NFA 定义的语言	11
3.6 NFA = DFA：两个模型的等价性	11
3.7 NFA 到 DFA 的转换：子集构造法	12
3.7.1 转换示例	12
4 课程总结	14

前言

本文是对《编译器导论》课程第三章第一部分 (Finite Automata) 的详细讲解笔记。原始讲义由南京大学计算机学院时清凯老师制作，共包含 124 页幻灯片。有限自动机是词法分析 (*Lexical Analysis*) 的理论基础，用于描述和识别正则语言 (*Regular Language*)。

本文按照原讲义的逻辑分为三个主要部分：

1. **第一部分：数学预备知识**—字母表、字符串、语言的操作与性质
2. **第二部分：确定型有限自动机 (DFA)**—DFA 的定义、语言识别、最小化、双向模拟等价性检验
3. **第三部分：非确定型有限自动机 (NFA)**—NFA 的定义、 ϵ -转移、NFA 到 DFA 的转换 (子集构造法)

有限自动机理论是编译原理中词法分析器的数学基础。理解 DFA 与 NFA 的关系，掌握 $NFA \rightarrow DFA$ 的转换方法，是实现词法分析器生成工具 (如 lex/flex) 的关键。

1 第一部分：数学预备知识 (Math Preliminaries)

在深入有限自动机之前，需要建立形式语言理论的基本概念。

1.1 字母表与字符串 (Alphabet and Strings)

核心概念

语言 (Language) 是字符串的集合，例如 $\{cat, dog, \dots\}$ 。

字符串 (String) 是定义在字母表 (Alphabet) 上的字母 (Letters) 序列。

- 字母表示例： $\Sigma = \{a, b, c, d, \dots, z\}$
- 字符串示例： cat, dog, apple, ...
- 小型字母表示例： $\Sigma = \{a, b\}$
 - 在此字母表上可构成的字符串： $a, b, ab, aab, aabb, \dots$
 - 语言是 $\{a, b, ab, aab, aabb, \dots\}$ 的任意子集

1.2 字符串操作 (String Operations)

1.2.1 连接 (Concatenation)

两个字符串的连接是将它们头尾相接：

$$w_1 = aabb; \quad w_2 = bbaa \implies w_1w_2 = aabbbbbaa$$

1.2.2 反转 (Reverse)

$$w = a_1a_2a_3a_4 \implies w^R = a_4a_3a_2a_1$$

1.2.3 长度 (Length)

字符串中字母的个数，记为 $|w|$ ：

$$w = a_1a_2a_3a_4 \implies |w| = 4$$

1.3 空字符串与子字符串

核心概念

空字符串 (Empty String) 记为 ϵ (epsilon), 是长度为 0 的特殊字符串。

$$|\epsilon| = 0$$

$$\epsilon aabb = aab\epsilon b = aabbe = aabb$$

子字符串 (Substring): 字符串中连续字符组成的子序列。例如 *abbab* 的子串包括 *abbab*, *abba*, *bbab*, *abb*, *bba*, ...

前缀与后缀 (Prefix and Suffix): 若 $w = uv$, 则 u 是前缀, v 是后缀。

- *abb* 的前缀: ϵ , a , ab , abb
- *abb* 的后缀: abb , bb , b , ϵ

1.4 幂、Kleene 星号与加号

核心概念

幂 (Power): $w^n = \underbrace{ww \dots w}_{n \text{ 次}}$; 特殊地, $w^0 = \epsilon$

Kleene 星号 (Kleene Star): Σ^* 表示由 Σ 中字母构成的所有可能字符串的集合 (包括 ϵ)

加号 (Plus): $\Sigma^+ = \Sigma^* - \{\epsilon\}$

示例:

- $(abb)^2 = abbabb$
- $(abb)^0 = \epsilon$
- $\Sigma = \{a, b\} \implies \Sigma^* = \{\epsilon, a, b, ab, abb, aab, \dots\}$
- $\Sigma^+ = \{a, b, ab, abb, aab, \dots\}$

笔记

重要结论: 一个语言就是 Σ^* 的任意子集。例如: $\{\epsilon\}$, $\{\epsilon, a, b\}$, $\emptyset$ 等等。

注意: $\{\epsilon\} \neq \emptyset$! 包含空字符串的集合不是空集。

1.5 语言上的操作 (Operations on Languages)

核心概念

语言本质上是集合，因此集合操作自然适用于语言。此外，基于字符串操作可以定义语言特有的操作。

1.5.1 集合操作

令 $L_1 = \{a, b\}$, $L_2 = \{a, ab\}$:

- 并集: $L_1 \cup L_2 = \{a, b, ab\}$
- 交集: $L_1 \cap L_2 = \{a\}$
- 差集: $L_1 - L_2 = \{b\}$
- 补集: $\bar{L} = \Sigma^* - L$
- 反转: $L^R = \{w^R : w \in L\}$

1.5.2 连接、幂与闭包

- 连接: $L_1 L_2 = \{xy : x \in L_1, y \in L_2\}$
- 幂: $L^n = \underbrace{LL \dots L}_{n \text{ 次}}$; 特殊地, $L^0 = \{\epsilon\}$
- 星闭包 (Star-Closure): $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$
- 正闭包 (Positive-Closure): $L^+ = L^1 \cup L^2 \cup \dots = L^* - \{\epsilon\}$

笔记

小测验:

1. $L = \{a, b\}$, 求 $L^2 = ?$
2. $L = \{a^n b^n : n \geq 0\}$, 求 $L^2 = ?$

答案:

1. $L^2 = \{aa, ab, ba, bb\}$
2. $L^2 = \{a^n b^n a^m b^m : n, m \geq 0\}$

2 第二部分：确定型有限自动机 (DFA)

2.1 有限自动机的直观理解

核心概念

有限自动机 (Finite Automaton) 是一种简单的计算模型：输入一个字符串，输出「接受 (Accept)」或「拒绝 (Reject)」。



图 1: 有限自动机的输入输出模型

以识别字符串”abba” 的自动机为例：自动机从初始状态出发，每读入一个字符就按照转移规则跳转到下一个状态。读完所有字符后，若停在终止状态 (Final State)，则接受该字符串；否则拒绝。

例如输入”abb” (而非”abba”)，读完后的状态不是终止状态，因此会被拒绝。

2.2 DFA 的形式化定义

核心概念

一个确定型有限自动机 (DFA, Deterministic Finite Automaton) 是一个五元组：

$$M = (Q, \Sigma, \delta, q_0, F)$$

五个组成部分：

- 1. Q ：有限状态集合 (A finite set of states)
- 2. Σ ：有限输入字母表 (A finite set of input characters / alphabet)
- 3. $\delta : Q \times \Sigma \mapsto Q$ ：转移函数 (Transition function)，如 $\delta(q, a) = q'$
- 4. $q_0 \in Q$ ：初始状态 (The start state)
- 5. $F \subseteq Q$ ：终止状态集合 (A finite subset of final states)

笔记

「确定型」的含义：对于任意状态 q 和任意输入字符 a ，转移函数 $\delta(q, a)$ 有且仅有唯一的下一个状态。在任何时刻，自动机的行为是完全确定的。

2.2.1 扩展转移函数

核心概念

扩展转移函数 $\delta^* : Q \times \Sigma^* \mapsto Q$ 将单字符转移扩展为字符串转移：

- $\delta^*(q, abba) = q'$ （读完整个字符串后到达的状态）
- $\delta^*(q, \epsilon) = q$ （空字符串不改变状态）

2.3 DFA 定义的语言

核心概念

设 DFA $M = (Q, \Sigma, \delta, q_0, F)$ ， M 所接受的语言定义为：

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$

即所有从初始状态出发、读完字符串后到达终止状态的字符串集合。

2.3.1 DFA 的实现：字符串检查器

Listing 1: 使用 DFA 检查字符串是否属于语言

```
void check(w, M) {
    q = q0;
    while (true) {
        c = read(w);
        if (c == None) {
            print(q in F ? "accept" : "reject");
            return;
        }
        switch (q) {
            case q0: if (c == 'a') { q = q1; } break;
            case q1: ... break;
            case q2: print("accept"); return; // q2 in F
            case ...
```



```
}  
}  
}
```

2.4 DFA 最小化 (DFA Minimization)

核心概念

DFA 最小化 的目标是减少 DFA 的状态数，从而提高计算效率。等价的 DFA 可以有不同的状态数，最小化可以找到状态数最少的等价 DFA。

2.4.1 最小化算法

核心思想：反复将状态划分为等价类 (*Equivalence Classes*)，使得同一等价类中的状态在任意输入下行为一致。

算法步骤 (以 6 状态 DFA 为例)：

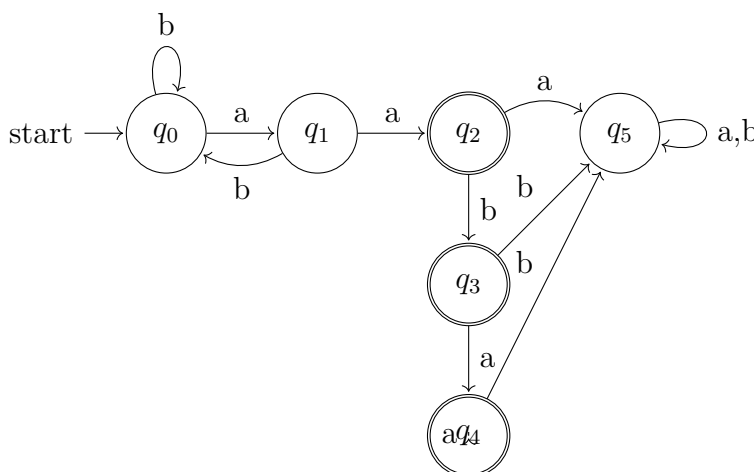


图 2: 一个 6 状态的 DFA (最小化前)

步骤	划分结果	说明
Step 1	$\{q_0, q_1, q_5\}, \{q_2, q_3, q_4\}$	区分终止状态和非终止状态
Step 2	$\{q_0, q_1\}, \{q_5\}, \{q_2, q_3, q_4\}$	$\delta(q_0, a/b)$ 和 $\delta(q_1, a/b)$ 在同一集合 但 $\delta(q_5, a/b)$ 不在
Step 3	$\{q_0, q_1\}, \{q_5\}, \{q_2, q_3, q_4\}$	结果不再变化，算法终止

表 1: DFA 最小化的划分过程

2.4.2 最小化结果

最小化后，等价的原始 DFA 变为只有 **3 个状态** 的 DFA: $q_{0,1}$ (合并 q_0, q_1), $q_{2\sim4}$ (合并 q_2, q_3, q_4), q_5 。

笔记

注意事项:

- 在执行上述划分步骤之前，需要先移除所有不可达状态 (*Unreachable States*)
- DFA 最小化的最优平均时间复杂度为 $O(n \log \log n)$ (Hopcroft 算法)

2.5 DFA 双向模拟 (DFA Bi-Simulation)

核心概念

DFA 双向模拟 (Bi-Simulation) 用于检查两个 DFA 的等价性，即验证 $L(M_1) = L(M_2)$ 。

基本原理: 对于任意输入字符串, M_1 到达终止状态当且仅当 M_2 到达终止状态。

方法: 构造状态对表，同时跟踪两个 DFA 在相同输入下的行为:

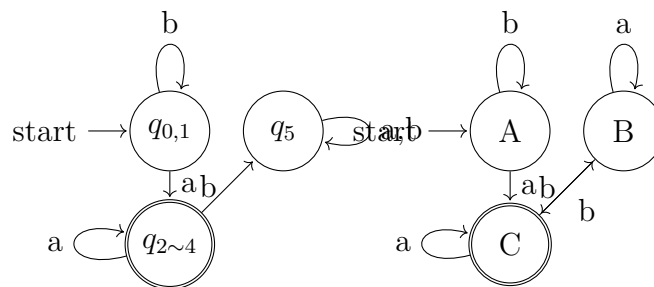


图 3: 两个待比较的 DFA

通过构造状态对 (q_i, s_j) ，跟踪两个 DFA 在每种输入下的共同行为。若对于所有可达的状态对, M_1 的状态在 F_1 中当且仅当 M_2 的状态在 F_2 中，则两个 DFA 等价。

3 第三部分：非确定型有限自动机 (NFA)

3.1 NFA 的直观理解

核心概念

非确定型有限自动机 (NFA, Non-deterministic Finite Automaton) 在每个状态下对于同一个输入字符，可能存在多个可选的状态转移。

非确定性的含义：

- 对于输入字符串 aa ，第一条字符 a 就可能有两种转移选择
- 如果某条路径无法接受字符串，需要回溯尝试其他路径
- 只要存在一条计算路径能够接受字符串，该字符串就被 NFA 接受

3.2 ϵ 转移 (ϵ -Transition)

核心概念

ϵ -转移 允许 NFA 在不消耗任何输入字符的情况下自发地转移到下一个状态。

这是 DFA 与 NFA 的一个重要区别——NFA 的转移函数接受 ϵ 作为输入。

3.3 NFA 的形式化定义

核心概念

一个非确定型有限自动机 (NFA) 是一个五元组：

$$M = (Q, \Sigma \cup \{\epsilon\}, \delta, q_0, F)$$

与 DFA 的关键区别在于转移函数：

- DFA:** $\delta : Q \times \Sigma \mapsto Q$ (确定性，唯一目标状态)
- NFA:** $\delta : Q \times (\Sigma \cup \{\epsilon\}) \mapsto 2^Q$ (非确定性，目标状态集合)

3.4 ϵ -闭包 (ϵ -Closure)

核心概念

ϵ -closure(q) 返回从状态 q 出发，通过零个或多个 ϵ -转移能到达的所有状态集合，包括 q 本身。

扩展转移函数 $\delta^* : Q \times \Sigma^* \mapsto 2^Q$ ：

$q' \in \delta^*(q, w)$ (其中 $w \in \Sigma^*$) 当且仅当：

1. 存在一条从 q 到 q'' 的路径消耗了字符串 w
2. 且 $q' \in \epsilon$ -closure(q'')

3.5 NFA 定义的语言

核心概念

回顾 DFA 定义的语言：

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$

NFA 定义的语言类似，但条件是到达的状态集合与 F 的交集非空：

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}$$

即存在至少一条路径能够到达某个终止状态。

3.6 NFA = DFA：两个模型的等价性

核心概念

NFA 与 DFA 的表达能力是等价的：

- 每个 DFA 天然就是一个 NFA (平凡情况)
- 每个 NFA 都可以转换为等价的 DFA (子集构造法)
- 即对于任意 NFA N ，都存在 DFA D ，使得 $L(N) = L(D)$

3.7 NFA 到 DFA 的转换：子集构造法

核心概念

核心思想： DFA 的每个状态对应于 NFA 的一个状态集合（子集）。通过模拟 NFA 在每种输入下可能到达的所有状态集合来构建 DFA。

算法步骤：

1. **Step 1:** 初始化 DFA，起始状态为 $\{q_0\}$ （NFA 初始状态的单元素集合）
2. **Step 2:** 对于 DFA 的每个状态 $\{q_i, q_j, \dots, q_k\}$ 和每个字母 $a \in \Sigma$ ，计算：

$$S = \bigcup_{q \in \{q_i, q_j, \dots, q_k\}} \delta^*(q, a)$$

3. **Step 3:** 在 DFA 中添加转移：

$$\delta_{\text{DFA}}(\{q_i, q_j, \dots, q_k\}, a) = S$$

若 S 中有任何状态是 NFA 的终止状态，则 S 是 DFA 的终止状态

3.7.1 转换示例

给定一个 NFA：

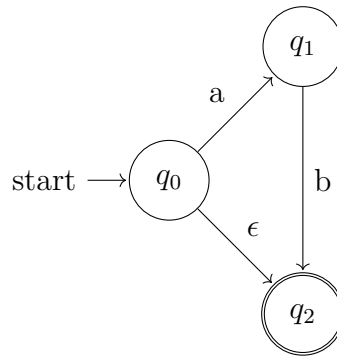


图 4: NFA 示例：识别 a 或 ab

转换过程：

- 初始 DFA 状态： $\{q_0\}$
- $\delta(\{q_0\}, a) = \{q_1, q_2\}$ （新建 DFA 状态）
- $\delta(\{q_1, q_2\}, a) = \emptyset$
- $\delta(\{q_1, q_2\}, b) = \{q_2\}$ （ $\{q_2\}$ 包含终止状态 $\Rightarrow$ 是 DFA 终止状态）

注意事项**NFA 转 DFA 的状态爆炸 (State Explosion):**

因为 DFA 的状态是 NFA 状态的子集，一个 n 状态的 NFA 在最坏情况下可能转换为最多 2^n 状态的 DFA。对于每个 n ，存在 n 状态的 NFA 使得其所有子集都可达，因此转换后的 DFA 恰好有 2^n 个状态，最坏时间复杂度为 $\Theta(2^n)$ 。

重要：将 NFA 转换为 DFA 时，不保证会得到规模更小的 DFA！

4 课程总结

本章作为词法分析的理论基础，涵盖了以下核心要点：

1. **形式语言基础：**字母表 Σ 、字符串、语言（ Σ^* 的子集）、字符串与语言上的各种操作（连接、幂、Kleene 星号、正闭包等）
2. **确定型有限自动机（DFA）：**
 - 五元组定义： $(Q, \Sigma, \delta, q_0, F)$
 - 语言定义： $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$
 - **DFA 最小化：**通过状态划分（初始划分终止/非终止状态，反复细化）减少状态数
 - **DFA 双向模拟：**通过跟踪状态对来检验两个 DFA 是否接受相同的语言
3. **非确定型有限自动机（NFA）：**
 - 允许多条转移路径和 ϵ -转移
 - 语言定义： $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset\}$
 - **NFA = DFA：**表达能力等价
 - **子集构造法：**将 NFA 转换为 DFA，最坏情况下产生指数级状态增长 $\Theta(2^n)$
4. **在编译器中的地位：**
 - DFA/NFA 定义的语言 = 正则语言（Regular Language）
 - 正则语言 $\leftrightarrow$ 正则表达式 $\leftrightarrow$ 有限自动机
 - 词法分析器（Lexer）正是基于此理论构造的

本文完

下一章将介绍正则表达式以及如何从正则表达式构建词法分析器。